

ANDROID DEVELOPMENT

B.E. (Electronics / E&T.C.) | 2019 Pattern | Semester VIII | Elective V

REGULAR END SEMESTER EXAM — 14 June 2023 | Max Marks: 70

Q1 a) Explain Android building block. [5 Marks]

The building blocks of an Android application are the core components from which all apps are constructed. Every Android app is made up of one or more of the following four components:

- **Activity** — Represents a single screen with a user interface. It is what the user sees and interacts with. For example, a messaging app might have one activity for the conversation list and another for reading a specific conversation.
- **Service** — A component that runs operations in the background without any user interface. Services are used for tasks like downloading files, playing music, or processing data even when the user is not actively looking at the app.
- **Broadcast Receiver** — A component that listens for and responds to system-wide broadcast messages. Android broadcasts events like 'battery is low', 'screen turned off', 'network connected', etc. Apps can register to receive any of these.
- **Content Provider** — A component that manages and shares a structured set of app data with other applications. It provides a standardized interface for reading and writing data. Examples include the Contacts app and Media Store.

All four components are declared in the `AndroidManifest.xml` file. Additional elements like `Intents` (for communication between components), `Resources` (layouts, images, strings), and the `AndroidManifest.xml` itself are also considered part of the overall application structure.

Q1 b) What is Intent? Explain its types with an example. [6 Marks]

What is an Intent?

An Intent is a messaging object in Android used to request an action from another app component. It acts as a courier between activities, services, or broadcast receivers. An Intent can be used to start a new activity, start a service, or deliver a broadcast. Besides specifying the action, Intents can also carry data using `putExtra()` key-value pairs.

Types of Intents

1. Explicit Intent

An Explicit Intent clearly specifies the exact target component (usually a specific Activity class) that should handle the request. This type is used primarily within the same application when you know exactly which component to start.

Example — Navigating from MainActivity to ProfileActivity: You create an Intent, specify `MainActivity.this` as the source and `ProfileActivity.class` as the destination. Optionally add extra data using `putExtra()`. Then call `startActivity()`. In ProfileActivity, retrieve the data using `getIntent().getStringExtra()`.

2. Implicit Intent

An Implicit Intent does not specify a particular component. Instead, it declares a general action to be performed, and Android searches all installed apps for one that can handle it. If multiple apps can handle it, Android shows a chooser dialog. If none can, the app crashes unless you handle that case.

Example — Opening a webpage: Create an Intent with `ACTION_VIEW` and set the data to a URI with an 'http' or 'https' scheme. Call `startActivity()`. Android opens the default browser (or prompts the user to choose one).

Example — Sharing text: Create an Intent with `ACTION_SEND`, set the type to 'text/plain', add the text as an extra, and call `startActivity` with a chooser. Android shows all apps that can share text (Gmail, WhatsApp, etc.).

Other Intent-related Concepts

- Intent Extras — Key-value pairs attached to an Intent to pass data between components (e.g., `putExtra('username', 'John')`).
- Intent Flags — Modify the behavior of how the activity is started (e.g., `FLAG_ACTIVITY_NEW_TASK`, `FLAG_ACTIVITY_CLEAR_TOP`).
- Pending Intent — A wrapped Intent that allows another app or the system to execute an action on behalf of your app at a later time (used in notifications and alarms).

Q1 c) What is an Activity? Draw lifecycle of it and explain. [6 Marks]

What is an Activity?

An Activity is a fundamental component in Android that provides a single screen with a user interface. It represents one thing the user can do, such as viewing a list, composing an email, or viewing a photo. Each activity is a separate Java class that extends `AppCompatActivity` (or `Activity`). Activities are declared in `AndroidManifest.xml` and typically have an associated XML layout file.

Activity Lifecycle

Android manages activities through a lifecycle — a series of states the activity passes through from creation to destruction. As the user navigates around the app, each activity transitions between states. Android calls specific callback methods at each transition, giving the developer an opportunity to respond appropriately.

State / Callback	Triggered When	What To Do
<code>onCreate()</code>	Activity is first created	Initialize the activity: call <code>setContentView()</code> , get view references with <code>findViewById()</code> , set up listeners
<code>onStart()</code>	Activity becomes visible but not yet interactive	Start visual animations or prepare content to show
<code>onResume()</code>	Activity is in the foreground and user can interact	Start camera, sensor, or media resources. Resume any paused operations.
<code>onPause()</code>	Another activity partially covers this one	Pause media, save user input, release resources that aren't needed when not in focus
<code>onStop()</code>	Activity is completely hidden (not visible)	Save data to database. Release heavy resources like animations.
<code>onRestart()</code>	Activity is coming back from stopped state to started	Restore any resources released in <code>onStop()</code>
<code>onDestroy()</code>	Activity is about to be destroyed permanently	Release all resources. Called before the activity is removed from memory.

Understanding the lifecycle is critical for: saving and restoring user data when the phone rotates or the user gets a phone call, releasing resources (like the camera or GPS) when not needed so other apps can use them, and ensuring the app doesn't crash or misbehave when interrupted.

Q2 a) Explain UI components. [5 Marks]

UI (User Interface) components are the visual building blocks used to construct Android app screens. They are all subclasses of the View class. They can be divided into several categories:

Text Components

- TextView — Displays read-only text. Used for labels, titles, descriptions.
- EditText — An editable text input field. Users can type in it. Has variants like email, password, phone, and number input types.
- AutoCompleteTextView — Extends EditText and shows suggestions as the user types.

Button Components

- Button — Standard rectangular clickable button.
- ImageButton — A button that displays an image instead of text.
- FloatingActionButton (FAB) — A circular button that floats above the UI, used for the primary action on a screen.

Selection Controls

- CheckBox — A two-state toggle (checked/unchecked) for multi-selection scenarios.
- RadioButton / RadioGroup — Used for single-choice selection. Only one RadioButton in a RadioGroup can be selected at a time.
- Switch — A two-state toggle switch for on/off settings.
- SeekBar — A horizontal slider that lets the user select a value from a range by dragging a thumb.
- Spinner — A dropdown widget for selecting one item from a list.

Display Components

- ImageView — Displays a bitmap image or drawable.
- ProgressBar — Shows loading progress (circular indeterminate or horizontal determinate).
- RatingBar — Displays and allows input of a star rating.

List and Data Components

- ListView — Displays a vertically scrollable list of items.
- GridView — Displays items in a two-dimensional scrollable grid.
- RecyclerView — A more efficient and flexible version of ListView, recommended for most list use cases.

Q2 b) Explain the structure of Android project. [6 Marks]

An Android project created in Android Studio follows a well-organized directory structure. Each folder and file has a specific purpose:

Top-level Structure

- app/ — The main module of the project. Contains all source code and resources.
- build.gradle (Project level) — Contains build configuration for the entire project.
- settings.gradle — Defines which modules are included in the project.
- gradle.properties — Contains project-wide Gradle settings.

Inside app/src/main/

- java/com.package.name/ — Contains all Java source code files, organized by package. Activities, adapters, helpers, and all other classes go here.
- res/layout/ — XML files that define the visual layout of each screen. One layout file per activity or fragment, typically.
- res/drawable/ — Image files (PNG, JPEG) and XML shape/selector drawables.

- res/values/ — strings.xml (all text used in the app), colors.xml (color palette), styles.xml (UI themes and styles).
- res/mipmap/ — Launcher icon files in multiple sizes for different screen densities (mdpi, hdpi, xhdpi, xxhdpi, xxxhdpi).
- res/menu/ — XML files that define menus (Options Menu, Context Menu, Popup Menu).
- AndroidManifest.xml — The most important configuration file. Declares all activities, services, receivers, permissions, app name, icon, and minimum SDK version.

app/build.gradle (Module level)

This file specifies the compileSdkVersion, minSdkVersion, targetSdkVersion, the applicationId (unique package name), and all library dependencies. This is where you add third-party libraries like Volley, Retrofit, or Glide.

Q2 c) What is Dalvik Virtual Machine? Compare with JVM. [6 Marks]

Dalvik Virtual Machine (DVM)

The Dalvik Virtual Machine is a custom virtual machine designed by Google specifically for running Android applications on mobile devices. Before Android 5.0 (Lollipop), every Android app ran on a DVM. Each app ran in its own DVM process, isolated from other apps for security.

DVM was built with mobile constraints in mind — limited RAM, slower CPUs, and battery limitations. It is register-based (uses registers for operations, like a real CPU) rather than stack-based. It runs .dex (Dalvik Executable) files which are a compact, mobile-optimized bytecode format derived from Java .class files using the dx conversion tool.

Note: From Android 5.0 onwards, DVM was replaced by ART (Android Runtime) which pre-compiles app code at install time (Ahead-of-Time compilation) for faster execution.

Comparison: DVM vs JVM

Feature	Dalvik VM (DVM)	Java VM (JVM)
Architecture	Register-based	Stack-based
Bytecode format	.dex files (Dalvik Executable)	.class files (Java bytecode)
Designed for	Mobile devices (ARM processors)	Desktop/server systems
Memory usage	Low — optimized for mobile	Higher — suited for systems with more RAM
Multiple instances	Yes — one per app simultaneously	Usually one shared instance
Execution	JIT (Just-in-Time) compilation	JIT compilation
Source	Google (Android-specific)	Oracle (originally Sun Microsystems)

Q3 a) Write a note on fragment. [5 Marks]

A Fragment is a reusable, modular portion of a user interface within an Activity. Think of a fragment as a 'sub-activity' — it has its own layout, its own Java class, and its own lifecycle, but it always lives inside a host Activity and cannot exist independently.

Why Use Fragments?

Fragments make it possible to build flexible, adaptive UIs that work across different screen sizes. On a phone, you might show a list in one full screen and details in another. On a tablet, both can be shown side by side in the same Activity using two fragments.

Fragment Lifecycle

The fragment lifecycle is similar to the activity lifecycle but has additional steps: `onAttach()` (connected to Activity), `onCreate()` (initialized), `onCreateView()` (layout inflated), `onViewCreated()` (views ready to use), `onStart()`, `onResume()`, `onPause()`, `onStop()`, `onDestroyView()` (layout removed), `onDestroy()`, and `onDetach()` (disconnected from Activity).

Adding Fragments

Fragments can be added to an activity statically by placing a fragment tag directly in the activity's XML layout. Or they can be added dynamically at runtime using `FragmentManager` and `FragmentTransaction` in Java code. Dynamic fragments are much more flexible — they can be added, removed, replaced, and stacked using the back stack.

Communication

Fragments communicate with their host Activity through an interface. The Activity communicates with a Fragment by passing data in a `Bundle` via `setArguments()`. Two fragments do not communicate directly — they always go through the Activity.

Q3 b) Which are the different components of screen? Explain each. [6 Marks]

The Android screen is composed of several layered components:

1. View

A View is the fundamental building block of the Android UI. Every visible element on the screen is a View or a subclass of View. A View occupies a rectangular area on the screen and is responsible for drawing itself and handling touch events. Basic views include `TextView`, `Button`, `ImageView`, `EditText`, `CheckBox`, `RadioButton`, `SeekBar`, and `ProgressBar`.

2. ViewGroup (Layouts)

A ViewGroup is a special type of View that acts as a container for other Views. It does not display anything itself but controls the arrangement and positioning of its child views. Common ViewGroups include `LinearLayout` (arranges children in a horizontal row or vertical column), `RelativeLayout` (positions children relative to each other or the parent), `ConstraintLayout` (uses constraints to position views — the most flexible and recommended), and `FrameLayout` (stacks views on top of each other, commonly used to host fragments).

3. Window

Every Activity has exactly one Window. The Window occupies the full screen (or a portion of it in multi-window mode) and provides the surface on which all Views are drawn. The Window sits between the app and the Android system and handles system UI elements like the status bar and navigation bar.

4. Surface

A Surface is a raw drawing area maintained by the system compositor. Regular Views draw into a Surface that is managed by the system. For performance-critical cases like camera preview or game rendering, `SurfaceView` provides direct access to a dedicated Surface with its own drawing thread, separate from the UI thread.

5. Canvas

A Canvas is a drawing interface that provides methods for drawing shapes, text, bitmaps, and more. When you create a custom View and override the `onDraw()` method, Android passes a Canvas object to you. You use the Canvas methods — `drawCircle()`, `drawRect()`, `drawText()`, `drawLine()`, `drawBitmap()` — along with a Paint object that defines the visual style (color, stroke width, anti-aliasing).

Q3 c) What is the use of Intent in activities? [6 Marks]

Intents serve several important purposes when working with activities:

1. Starting a New Activity

The most common use is to navigate from one activity to another. You create an Intent specifying the source context and destination Activity class, then call `startActivity()`. This launches the new activity and places it on the activity back stack. The user can press the back button to return to the previous activity.

2. Passing Data Between Activities

Intents can carry data using the `putExtra()` method. You attach key-value pairs of any primitive type (String, int, boolean, etc.) or Serializable/Parcelable objects. The receiving activity retrieves this data by calling `getIntent()` to get the Intent that started it, then calling `getStringExtra()`, `getIntExtra()`, etc.

3. Getting a Result from an Activity

When you need the launched activity to return a result, use `startActivityForResult()`. For example, launching a gallery activity to pick an image and receive the selected image URI back. The launched activity calls `setResult()` with a result code and optional data, then `finish()`. The calling activity receives the result in the `onActivityResult()` callback.

4. Starting Built-in App Activities

Implicit intents let your app launch activities from other installed apps. Common examples include: opening a URL in a browser using `ACTION_VIEW`, composing an email using `ACTION_SENDTO`, calling a phone number using `ACTION_CALL`, opening the camera to take a photo using `MediaStore.ACTION_IMAGE_CAPTURE`, and opening the gallery to pick a file.

5. Intent Flags

Flags modify the behavior of how the activity is launched. For example, `FLAG_ACTIVITY_NEW_TASK` starts the activity in a new task, and `FLAG_ACTIVITY_CLEAR_TOP` removes all activities above the target in the stack, reusing the existing instance.

Q4 a) Explain Toast. [5 Marks]

Toast is a simple, non-intrusive way to display a brief message to the user. It appears as a small popup at the bottom of the screen (by default), stays visible for a short time, and then automatically disappears without any user interaction required. Toasts do not receive focus and don't interrupt the user's current task.

Characteristics of Toast

- Non-interactive — the user cannot click on a Toast; it just shows information.
- Automatically disappears — it does not stay on screen permanently.
- Duration options — `Toast.LENGTH_SHORT` (approximately 2 seconds) or `Toast.LENGTH_LONG` (approximately 3.5 seconds).
- Does not block UI — the app remains fully functional while the Toast is visible.

Creating a Toast

A Toast is created using `Toast.makeText()` with three arguments: the context (usually the activity), the message text, and the duration. Then call `.show()` to display it. You can create it from an `onClick` listener, after a database operation, after a network call, or whenever you want to briefly inform the user about something.

Custom Toast

Android also supports custom Toasts where you define your own layout XML file and inflate it as the Toast's view using `setView()`. This allows you to show an icon alongside the message, change the background color, or display a more complex layout. However, custom Toasts are deprecated from Android 11 onwards when the app is in the background.

When to Use Toast vs Other Notifications

Use Toast when	Use Snackbar when	Use Dialog when
Brief, simple message	Action can be undone (e.g., Undo delete)	User input is required
No user action needed	Message related to current screen	Critical confirmation needed
Non-critical information	Requires user acknowledgement	App cannot proceed without response

Q4 b) Explain lifecycle of fragment. [6 Marks]

Just like an Activity has a lifecycle, a Fragment also has a lifecycle. The fragment lifecycle is closely tied to its host Activity's lifecycle — when the Activity is paused, all its fragments are paused too.

Callback	When It Is Called	What To Do Here
<code>onAttach()</code>	Fragment is associated with its host Activity	Get a reference to the Activity; cast and store the listener interface
<code>onCreate()</code>	Fragment is being created (non-UI initialization)	Initialize variables, retrieve arguments from <code>getArguments()</code>
<code>onCreateView()</code>	Time to create and return the fragment's UI layout	Inflate the fragment's layout XML and return the root View
<code>onViewCreated()</code>	The view returned by <code>onCreateView()</code> is fully ready	Get references to views with <code>getView().findViewById()</code> . Set up click listeners.
<code>onStart()</code>	Fragment becomes visible to the user	Start lightweight animations or prepare content

onResume()	Fragment is fully interactive in the foreground	Register sensors, start camera, resume media
onPause()	Fragment is partially hidden or losing focus	Pause media, save user data, release sensors
onStop()	Fragment is no longer visible	Release heavy resources, save state
onDestroyView()	The view hierarchy associated with the fragment is being removed	Clean up view binding references to avoid memory leaks
onDestroy()	Fragment instance is being cleaned up	Release all remaining resources
onDetach()	Fragment is dissociated from its host Activity	Set Activity reference to null

Q4 c) How to call built-in application using Intent? [6 Marks]

Android's implicit Intent system allows any app to launch built-in system applications without knowing their internal code. You simply declare the action and data, and Android finds the right app to handle it.

Common Built-in App Calls

- Open a Web URL — Create an Intent with ACTION_VIEW and set the data to a Uri parsed from an 'http://' or 'https://' URL. Calling startActivity() opens the default web browser.
- Make a Phone Call — Use ACTION_CALL with a 'tel:' URI containing the phone number. Requires CALL_PHONE permission. For a safer approach, use ACTION_DIAL which opens the dialer without automatically placing the call.
- Send an Email — Use ACTION_SENDTO with a 'mailto:' URI. Set EXTRA_EMAIL for recipients, EXTRA_SUBJECT for the subject, and EXTRA_TEXT for the body. Starts the default email client.
- Send a Text Message (SMS) — Use ACTION_SENDTO with an 'smsto:' URI and set EXTRA_SMS_BODY for the message content. Opens the default SMS app.
- Capture a Photo — Use ACTION_IMAGE_CAPTURE from MediaStore. Use startActivityForResult() to get the captured photo back as a Bitmap in the onActivityResult() callback.
- Pick an Image from Gallery — Use ACTION_PICK with MediaStore.Images.Media.EXTERNAL_CONTENT_URI. Use startActivityForResult() to receive the selected image's URI.
- Open Maps at a Location — Use ACTION_VIEW with a 'geo:latitude,longitude' URI. Opens Google Maps centered at the specified coordinates.
- Open Settings — Use Settings.ACTION_SETTINGS intent. Opens the device settings screen.

Safety Check

Before calling startActivity() with an implicit intent, it is good practice to verify that some app exists to handle it. Use the resolveActivity() method on the Intent with the PackageManager. If the result is null, no app can handle the intent and you should handle that gracefully to avoid an app crash.

Q5 a) What is View? Explain any two views. [6 Marks]

What is a View?

In Android, a View is the basic building block of the user interface. Every element that the user can see and interact with on the screen is a View or a subclass of View. A View represents a rectangular area on the screen and is responsible for drawing its own content and handling user input events like touches and clicks. Views are arranged in a hierarchy called the View tree, where ViewGroups (layouts) contain other Views or more ViewGroups.

View 1 — EditText

EditText is one of the most important and commonly used views. It is an extension of TextView that allows the user to enter and edit text. It displays a text field with a cursor and brings up the soft keyboard when the user taps on it.

Key characteristics of EditText include: the `inputType` attribute which controls what type of data can be entered and what keyboard appears (text, number, email, password, phone, date), the `hint` attribute which shows placeholder text when the field is empty, the `maxLength` attribute for restricting input length, and the `imeOptions` attribute for customizing the Enter/Done button on the keyboard.

In Java, you retrieve the text using `getText().toString()` and set text using `setText()`. You can add a `TextWatcher` to monitor text changes in real time.

View 2 — ImageView

ImageView is used to display images in an Android app. It can display images from local resources (`res/drawable/`), from files on the device, from a URL (using libraries like Glide or Picasso), or from a `Bitmap` object in memory.

Key attributes include: `src` (sets the image), `scaleType` (controls how the image is scaled and positioned within the view — options include `centerCrop`, `fitCenter`, `fitXY`, `centerInside`), `adjustViewBounds` (adjusts the view's size to match the aspect ratio of the image), and `tint` (applies a color filter to the image).

In Java, you set the image using `setImageResource()` for a drawable resource ID, `setImageBitmap()` for a `Bitmap` object, `setImageDrawable()` for a `Drawable`, or `setImageURI()` for a content or file URI.

Q5 b) What is use of Toggle Button, Radio Button and Radio Group Views? [6 Marks]

Toggle Button

`ToggleButton` is a two-state button that the user can toggle between an ON state and an OFF state. It displays different text for each state — by default 'ON' when active and 'OFF' when inactive — but you can customize both using the `textOn` and `textOff` attributes. It is typically used for enabling or disabling a feature, like turning notifications on or off, enabling dark mode, etc.

In Java, check the current state using `isChecked()` which returns true when the button is in the ON state. Respond to state changes using `setOnCheckedChangeListener()` which provides the button view and the new boolean `isChecked` value.

Radio Button

A `RadioButton` is a two-state button that can be either checked (selected) or unchecked. Unlike a `CheckBox`, `RadioButtons` are designed to be used in groups where only one option can be selected at a time. When used alone, a `RadioButton` behaves like a `CheckBox`.

The key characteristic is exclusivity — when used inside a `RadioGroup`, selecting one `RadioButton` automatically deselects all other `RadioButtons` in the same group. Check the state using `isChecked()` or by checking which `RadioButton`'s ID matches `getCheckedRadioButtonId()`.

RadioGroup

`RadioGroup` is a special `ViewGroup` container designed to hold `RadioButtons`. Its primary job is to enforce mutual exclusion — only one `RadioButton` within a `RadioGroup` can be selected at any time. When the user selects one `RadioButton`, `RadioGroup` automatically unchecks all others.

Use `getCheckedRadioButtonId()` to find the ID of the currently selected `RadioButton`. Use `setOnCheckedChangeListener()` to be notified when the selection changes. Use `clearCheck()` to deselect all `RadioButtons` in the group.

A common use case is a gender selection field (Male / Female / Other) or a payment method selector (Credit Card / Debit Card / UPI) where only one option should be selected.

Q5 c) What is Fade In / Fade Out Animation? Explain it. [6 Marks]

Fade In and Fade Out animations are visual effects that change the transparency (alpha value) of a View over time. They belong to the View Animation (Tween Animation) system in Android.

Fade In Animation

Fade In gradually increases a View's opacity from fully transparent (invisible) to fully opaque (fully visible). The alpha value goes from 0.0 (completely transparent) to 1.0 (completely opaque) over a specified time duration. The effect makes the view appear to smoothly materialize on screen rather than popping in abruptly.

Fade Out Animation

Fade Out is the opposite — it gradually decreases a View's opacity from fully visible to fully transparent. The alpha value goes from 1.0 to 0.0 over time. The view appears to smoothly vanish. Note that after the animation ends, the View is still in the layout and takes up space unless you explicitly set its visibility to GONE.

Implementation

Fade animations are created using the alpha XML tag in an animation file saved in `res/anim/`. Specify `fromAlpha` (starting transparency) and `toAlpha` (ending transparency), along with the duration in milliseconds. You can also add a `startOffset` to delay when the animation begins. Load the animation using `AnimationUtils.loadAnimation()` and start it by calling `view.startAnimation()`.

Android also provides built-in fade animations: `android.R.anim.fade_in` and `android.R.anim.fade_out`, which are commonly used in Fragment transactions and ImageSwitcher transitions.

Common Uses

- Fading in a splash screen logo when the app starts.
- Fading out a loading overlay once content has loaded.
- Creating smooth transitions in an ImageSwitcher gallery.
- Fading in error messages or success overlays.

Q6 a) Explain Auto Complete Text View and Picker views. [6 Marks]

AutoCompleteTextView

AutoCompleteTextView is a subclass of EditText that shows a dropdown list of suggestions as the user types. It helps users complete their input faster and reduces typing errors. When the user types a few characters, the view shows a list of possible completions from a predefined dataset. The user can tap a suggestion to complete the input.

It is set up using an ArrayAdapter which maps the suggestions data to the list item views in the dropdown. The completionThreshold attribute (default value: 2) controls how many characters the user must type before suggestions start appearing. You can also use setDropDownHeight() and setDropDownWidth() to control the size of the suggestion dropdown.

Use cases include search bars, city/country name fields, email address fields (using previously used addresses), and any field where there is a known set of valid inputs.

Date Picker

DatePicker is a UI widget that lets the user select a date by choosing the day, month, and year. It can be displayed as a full calendar view (calendarViewShown=true) or as a compact spinner-style picker (calendarViewShown=false). It is typically used in forms where a date is required, like a date of birth or appointment date. The selected date is retrieved using getDayOfMonth(), getMonth(), and getYear().

For a better user experience, use DatePickerDialog which presents the date picker inside a popup dialog rather than occupying space permanently in the layout.

Time Picker

TimePicker is a UI widget that lets the user select a time — specifically hours and minutes. It can be displayed in 12-hour format (with AM/PM selection) or 24-hour format (set using setIs24HourView(true)). Like DatePicker, it is typically shown as a TimePickerDialog for a better user experience. Use getHour() and getMinute() to retrieve the selected time.

Q6 b) How to add audio and video in android project? [6 Marks]

Adding Audio

Android provides the MediaPlayer class for playing audio files. To add audio to a project, place the audio file (MP3, WAV, OGG) in the res/raw/ folder. To play it, create a MediaPlayer instance using MediaPlayer.create() with the context and the resource ID of the audio file. Call start() to begin playback, pause() to pause, and stop() followed by release() to stop and free resources.

For more control over audio, the SoundPool class is better for short sound effects (like UI sounds or game effects) because it pre-loads sounds into memory for very fast, low-latency playback. For recording audio, use the MediaRecorder class which can record from the microphone and save to a file.

Required permission for recording: RECORD_AUDIO. For accessing external audio files: READ_EXTERNAL_STORAGE (on older Android versions).

Adding Video

For playing video, Android provides the VideoView widget which is the simplest approach. Add a VideoView to your layout XML. In Java, set the video source using setVideoURI() (for a URI pointing to a video file) or setVideoPath() (for a file path string). Call start() to play. Add a MediaController by creating a new MediaController object and calling setMediaController() to attach transport controls (play, pause, seek bar) to the VideoView.

For more advanced video features, use the ExoPlayer library which is Google's recommended media player. It supports more formats, adaptive streaming (HLS, DASH), and provides more customization. Add it as a dependency in build.gradle.

Video files should be placed in the res/raw/ folder for bundled resources, but large video files are more typically loaded from a URL or from device storage.

Q6 c) Explain 2D and 3D Graphics. [6 Marks]

2D Graphics

2D graphics are flat, two-dimensional visuals. Android provides several methods:

- **Canvas and Paint** — Create a custom View class and override `onDraw()`. The Canvas provides drawing methods like `drawCircle()`, `drawRect()`, `drawText()`, `drawLine()`, `drawOval()`, and `drawBitmap()`. The Paint object controls color, stroke width, text size, typeface, and anti-aliasing.
- **XML Shape Drawables** — Define shapes in XML files in `res/drawable/` using shape, solid, stroke, corners, and gradient tags. No image files needed; scales perfectly.
- **SurfaceView** — For smooth, high-performance 2D animation (games, live graphics). Draws on a separate thread outside the main UI thread.

3D Graphics

3D graphics involve rendering three-dimensional objects with depth, perspective, and lighting.

Android uses OpenGL ES for this:

- **GLSurfaceView** — A View that provides an OpenGL rendering surface. Manages the OpenGL context and a dedicated background rendering thread automatically.
- **GLSurfaceView.Renderer** — An interface you implement with three methods: `onSurfaceCreated()` for one-time initialization (set background color, load shaders), `onSurfaceChanged()` for handling size changes (set viewport), and `onDrawFrame()` called every frame to draw the scene.
- **OpenGL ES 2.0 and 3.0** — Provides programmable shaders (vertex and fragment shaders written in GLSL) for custom lighting, textures, and effects.

For simpler 3D needs, third-party libraries like SceneView (based on ARCore and Filament) or jMonkeyEngine provide higher-level APIs.

Q7 a) Write difference between Internal storage and External storage. [6 Marks]

Aspect	Internal Storage	External Storage
Definition	Private storage built into the device for app-specific files	Shared storage accessible by users and possibly other apps
Access by other apps	Not accessible — completely private	Accessible by other apps with proper permission
Access by user	Not directly accessible (no root)	Directly visible and browsable by user
Permissions required	None	READ/WRITE_EXTERNAL_STORAGE for Android < 10
Availability	Always available	May be unavailable (SD card removed)
File deletion on uninstall	All files deleted	App-specific folder deleted; other files remain
Typical use	Private app data, databases, settings	Downloads, user media, files to share
Storage location	/data/data/packageName/files/	/sdcard/Android/data/packageName/ or /sdcard/
Storage size	Limited by device internal memory	Usually larger — especially with SD card

Q7 b) Write a note on SQLite Databases. [6 Marks]

SQLite is a self-contained, serverless, zero-configuration, transactional SQL database engine that is embedded in Android. Every Android device has SQLite built in.

Key Features

- Serverless — No database server to install or configure. The library links directly into the app.
- Single file — The entire database (tables, indexes, data) is stored in one .db file in /data/data/packageName/databases/.
- Standard SQL — Uses familiar SQL syntax: CREATE TABLE, INSERT INTO, SELECT, UPDATE, DELETE, DROP TABLE.
- ACID transactions — Guarantees that operations are atomic, consistent, isolated, and durable — protecting data integrity.
- Small footprint — Typically less than 500KB, perfect for mobile devices.

SQLiteOpenHelper

The proper way to use SQLite in Android is through SQLiteOpenHelper. You create a subclass with a constructor that sets the database name and version. The onCreate() method creates the initial database schema (tables). The onUpgrade() method handles version changes — called when you increment the version number to modify the schema while preserving existing data.

CRUD Operations

Create: Use getWritableDatabase() and ContentValues with db.insert() to add rows. Read: Use getReadableDatabase() and db.rawQuery() with a SELECT statement; iterate results with a Cursor. Update: Use db.update() with ContentValues and a WHERE condition. Delete: Use db.delete() with a WHERE condition.

Always close the Cursor and the database connection after use to prevent memory leaks. For large datasets, perform operations in a background thread (AsyncTask, Thread, or modern alternatives like Coroutines or LiveData with Room).

Q7 c) How to create, open and close database? [6 Marks]

Creating a Database

A database in Android is created automatically when you first call `getWritableDatabase()` or `getReadableDatabase()` on an `SQLiteOpenHelper` instance. If the database file does not already exist on the device, Android creates it and calls the `onCreate()` method where you write your `CREATE TABLE` statements. If the database already exists, it is simply opened.

Opening a Database

To open a database for writing (inserts, updates, deletes), call `getWritableDatabase()`. This returns an `SQLiteDatabase` object with both read and write capabilities. To open for reading only, call `getReadableDatabase()`. This is more efficient as it may open a read-only connection if the database is at capacity. Typically you create the `SQLiteOpenHelper` once (usually in the Activity's `onCreate` or as a singleton) and keep the reference.

Closing a Database

After all database operations are complete, you should close the `SQLiteDatabase` object by calling `db.close()`. If you used a `Cursor` to read data, always close it first with `cursor.close()` before closing the database. If the `SQLiteOpenHelper` itself is no longer needed, you can close it with `helper.close()` which closes the underlying database connection.

It is good practice to open the database as late as possible, perform operations quickly, and close it as soon as you are done. Holding the database open for a long time or forgetting to close it can cause memory leaks and issues with other components trying to access the database.

Q8 a) Explain JSON Web Service. [6 Marks]

What is a JSON Web Service?

A JSON Web Service is a REST (Representational State Transfer) API that communicates over HTTP and uses JSON (JavaScript Object Notation) as the data format for requests and responses. Mobile apps use web services to fetch data from remote servers — like weather information, product lists, news articles, or user profiles — and to send data back to the server.

How It Works

The Android app sends an HTTP request (typically GET to retrieve data or POST to send data) to a server URL called an endpoint. The server processes the request and responds with a JSON string. The app then parses this JSON string into Java objects using Android's JSON parsing classes.

Making an HTTP Request

All network operations must be performed on a background thread because they can take time and would block the UI thread, causing the app to become unresponsive. Android will throw a `NetworkOnMainThreadException` if you attempt network calls on the main thread. The `INTERNET` permission must be declared in `AndroidManifest.xml`.

The Volley library (developed by Google) simplifies network calls significantly. You create a `RequestQueue` and then create a `JsonObjectRequest` or `JsonArrayRequest` with the URL, a success listener, and an error listener. Volley automatically handles threading, caching, and retries. For even more power and flexibility, the Retrofit library (by Square) is widely used in production Android apps.

Parsing the JSON Response

The JSON response is typically a `JSONObject` (for a single item) or a `JSONArray` (for a list of items). Use `getString()`, `getInt()`, `getBoolean()`, `getDouble()` to extract values. Use `getJSONObject()` or `getJSONArray()` for nested structures. Wrap parsing in try-catch to handle `JSONException` errors for missing keys or malformed data.

Q8 b) How to access phone services like Call, SMS, MMS in Android application? [6 Marks]

Making Phone Calls

To initiate a phone call from an app, use an Intent with `ACTION_CALL` and a 'tel:' URI containing the phone number. This requires the `CALL_PHONE` permission in the manifest and runtime permission from the user. A safer alternative that doesn't require permission is `ACTION_DIAL`, which opens the phone dialer with the number pre-filled but waits for the user to press Call manually.

Sending SMS

There are two approaches for sending SMS: Using an Intent with `ACTION_SENDTO` and an 'smsto:' URI opens the default SMS app with the number and message pre-filled, letting the user send it. No permission is needed for this. Programmatic sending uses the `SmsManager` class. `SmsManager.getDefault()` gives the manager object. Use `sendTextMessage()` for short messages and `sendMultipartTextMessage()` for long messages that exceed 160 characters. This requires the `SEND_SMS` permission.

Sending MMS

MMS (Multimedia Messaging Service) allows sending messages with images, audio, or video attachments. Sending MMS programmatically is complex. The standard approach is to use an Intent (`ACTION_SEND` or `ACTION_SENDTO`) to open the default messaging app and attach the media content using `putExtra()` with the media URI. This requires no special permissions as it delegates to the system messaging app.

Monitoring Calls and SMS

To monitor incoming calls, you can listen for the `TelephonyManager.ACTION_PHONE_STATE_CHANGED` broadcast. Your `BroadcastReceiver` can detect incoming calls, ringing state, and hang-up events. The `READ_PHONE_STATE` permission is required. To receive incoming SMS, listen for the `Telephony.Sms.Intents.SMS_RECEIVED_ACTION` broadcast with the `RECEIVE_SMS` permission.

Q8 c) Write a note on Bluetooth / Wi-Fi Connectivity. [6 Marks]

Bluetooth Connectivity

Bluetooth allows short-range wireless communication between devices (typically up to 10 meters). Android's Bluetooth API is in the `android.bluetooth` package.

To use Bluetooth, add the `BLUETOOTH`, `BLUETOOTH_ADMIN` permissions to the manifest (and `BLUETOOTH_SCAN`, `BLUETOOTH_CONNECT` for Android 12+). Check if Bluetooth is supported using `BluetoothAdapter.getDefaultAdapter()`. If it returns null, the device does not have Bluetooth. Check if it's enabled using `adapter.isEnabled()`. If not, prompt the user to enable it using an Intent with `BluetoothAdapter.ACTION_REQUEST_ENABLE`.

To discover nearby devices, call `adapter.startDiscovery()`. Register a `BroadcastReceiver` for `BluetoothDevice.ACTION_FOUND` to receive discovered devices. Once you have a `BluetoothDevice`, create a connection using `BluetoothSocket` for classic Bluetooth or `BluetoothGatt` for Bluetooth Low Energy (BLE). Classic Bluetooth is used for continuous data streams (audio, file transfer). BLE is used for low-power sensor data (fitness trackers, heart rate monitors).

Wi-Fi Connectivity

Android provides the `WifiManager` class for working with Wi-Fi. Get it using `getSystemService(WIFI_SERVICE)`. Check the Wi-Fi state using `isWifiEnabled()`. Enable or disable Wi-Fi using `setWifiEnabled()` (deprecated in Android 10 — users must now control this manually through settings). Scan for available networks using `startScan()` and get results using `getScanResults()` which returns a list of `ScanResult` objects with network name (SSID), signal strength, security type, and frequency.

To connect to a specific network programmatically in older Android versions, create a `WifiConfiguration` with the SSID and password and call `addNetwork()` followed by `enableNetwork()`. In Android 10 and above, use the `WifiNetworkSpecifier` with a `NetworkRequest` for requesting a connection.

Required permissions include `ACCESS_WIFI_STATE`, `CHANGE_WIFI_STATE`, and `ACCESS_FINE_LOCATION` (required for scanning in Android 6.0+).